CS781 Capstone Project: In-House Legal Intelligence System

Rudra Dave
261110055
rudramdave26@iitk.ac.in
Indian Institute of Technology Kanpur (IIT Kanpur)

Code: https://github.com/Rudra5972/llm_capstone_261110055

Abstract

The main concern for a law firm using language models is confidentiality: its legal documents cannot be sent to outside services. This project therefore develops an in-house legal NLP system using only the official competition data. We built a byte-level BPE tokenizer with a 32k-token vocabulary and trained four decoder-only models, each with 124–128M parameters, in PyTorch. The architectures follow GPT-2, Llama, Mistral with sliding-window attention, and BLOOM with ALiBi. The Phase-1 corpus contains 4.18B tokens, but the 9-hour Kaggle limit allowed each model to train on roughly 0.36–0.39B tokens. We then adapted all four models to six legal tasks: named entity recognition, rhetorical-role prediction, judgment prediction, statute identification, summarization, and prior-case retrieval. We tested full, partial, and parameter-efficient fine-tuning alongside knowledge distillation, and used bidirectional attention for the encoder tasks. All four foundations finished with similar validation loss (1.85–1.89 nats/token). At equal token counts, ALiBi improved loss by 0.05 nats over learned absolute positions, while the 512-token sliding window made no measurable difference.

1 Competition Result

Codabench username: r_261110055 **Roll number:** 261110055 **Total valid Codabench**

Table 1: Official phase scores and ranks (best valid submission per phase).

Phase	Task	Primary metric (value)	Rank	Valid submissions
1	Legal LM Pretraining	BERTScore-F1 0.8287	94	4
2	Legal NER	Strict Macro-F1 0.5325	61	22
3	Rhetorical role prediction	Macro-F1 0.4692	87	19
4	Court judgment prediction	Macro-F1 0.7348	60	7
5	Legal statute identification	Macro-F1 0.3791	60	7
6	Abstractive summarization	BERTScore-F1 0.8448	44	7
7	Prior Case Retrieval (PCR)	Micro-F1@7 0.3952	64	7

submissions across all phases: 73 **Date ranks were recorded:** 9/11/2026
The minimum of four distinct submissions was met in every phase (Phase 1: 4, Phase 2: 22, Phase 3: 19, Phases 4–7: 7 each; 73 in total). Phase 1 has the fewest submissions because each one required a full 9-hour pretraining run.

2 Introduction and Problem Description

Legal documents often contain confidential information, so a law firm cannot simply upload them to a commercial language-model API. The practical aim of this project is to build and run the models locally. The capstone evaluates that idea through seven tasks on Indian legal text, using foundation models trained from scratch rather than downloaded pretrained weights.

Phase 1 is legal text continuation: given 512 words, the model generates the next 128 words. Its main metric is BERTScore-F1, with ROUGE-L and BLEU-4 as secondary metrics. Phase 2 is legal named entity recognition (L-NER). It identifies character spans (s, e, ℓ) for 12 types; strict macro-F1 counts a prediction only when both offsets and the label are correct. Phase 3 assigns one of 13 rhetorical roles to each sentence in Corporate-Law (CL) and IT judgments. Phase 4 predicts a judgment's binary outcome, while Phase 5 identifies the statutes cited in a case as a multi-label task. Both use macro-F1 as the main metric; Phase 5 also reports micro-F1 and subset accuracy. Phase 6 produces an abstractive summary, evaluated mainly by BERTScore-F1 and also by ROUGE-1/L. Phase 7 returns seven earlier cases relevant to a query case from a fixed candidate set, measured by micro-F1@7.

The project had five practical limits: models had to stay within 50M–300M parameters; all layers, tokenizers, and adaptation methods had to be implemented in PyTorch without pretrained weights or prebuilt model classes; external legal text was not allowed; training had to run on free Kaggle hardware (two T4 GPUs, 12-hour sessions and 30 GPU-hours per week); and several metrics were particularly strict about small errors. These limits shaped the comparisons. We asked whether positional embeddings or attention windows matter under equal compute, whether a causal model can be used for classification with bidirectional attention, which fine-tuning methods work for each task, and how to handle judgments longer than the 1024-token context. We completed Phases 1–7, including data checks and validated prediction.json submissions. Phase 8 (Final Evaluation), external datasets, and human legal review were outside this work. Across the phases, our workflow was to check the supplied data, train or adapt the model, compare predictions on development examples, and validate the submission format before uploading. This kept the process consistent even though the tasks used different labels, outputs and evaluation metrics.

3 Data Analysis and Preprocessing

3.1 Sources and official splits

We used only the official Exploration-Lab/CS-781-Capstone dataset and its phase-specific splits. Phase 1 contains 3,480,220 documents from four sources: Acts (0.73 GB), Supreme Court (1.32 GB), Tribunals (6.48 GB), and High Courts (10.67 GB). Its test set has 1,000 prompts. We reserved 1% of the documents for validation (41.98M tokens). We also sampled every 500th document to build a local prompt–continuation test set, which yielded 47 usable pairs.

For Phase 2, there are three folds of 35 judgments. The first two contain 8,393 labelled entities, and the third is held out for testing. We split the 70 labelled judgments by document into 60 training cases and 10 development cases, using seed 42. All labels were checked to be present in the development split. Phase 3 supplies separate train, development, and test data for CL and IT judgments with 13 rhetorical roles. Phases 4–7 also provide JSON train/dev/test files. Each run records the corresponding dataset sizes, labels, text lengths, and test overlap in its data audit.json file, which is included with the code.

3.2 Corpus statistics that actually informed design

Phase 1. By bytes the corpus is 55.6% High Courts (10.67 GB), 33.7% Tribunals (6.48 GB), 6.9% Supreme Court and 3.8% Acts; High Court documents are short (roughly 900 tokens on average)

while Acts and Supreme Court cases are usually several thousand tokens long. We kept the original distribution instead of balancing the sources, since the hidden prompts are sampled from the same collection. After tokenization, the training corpus contains 4.18B tokens. Even 12,000 steps use only 0.39B tokens, or about 9% of the corpus, so reusing the same text was not the main issue. The real problem was training time: each model saw roughly three tokens per parameter, well below the often-used target of about 20.

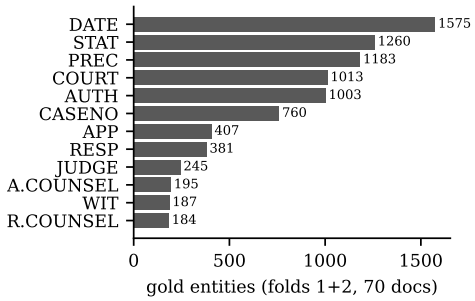


Figure 1: Phase-2 gold entities per label in folds 1–2. Counsel and witness labels are 5–8 times rarer than DATE or STAT.

Phase 2. Figure 1 makes the class imbalance clear. DATE has 1,575 examples, while R.COUNSEL, WIT, and A.COUNSEL each have fewer than 200. Most entities are short: the median length is 4 BPE tokens, the 99th percentile is 25, and the maximum is 54. Judgments, however, can reach 24,090 tokens, with about 230k tokens in a fold. This is why we split the text into windows rather than processing an entire judgment at once. We found no nested or crossing entities, so standard IOB2 labels were sufficient.

Phases 3–7. Each training run starts by recording dataset sizes, label counts, lengths, and any test-set overlap in data audit.json. These checks guided the individual task designs. For Phase 3, we kept all 13 roles and tuned domain-specific class biases for rare labels such as Dissent instead of merging them. For Phase 4, we used square-root inverse class weights and tuned the decision threshold because macro-F1 treats both outcomes equally. In Phase 5, there is one sigmoid output for every statute found in training; an unseen statute therefore cannot be predicted, which limits recall for rare statutes. We also gave extra attention to the end of judgments in Phase 4 because the final outcome is often stated there. In practice, the dataset checks helped us choose the window lengths, class weighting, and prediction rules for each task instead of treating all six downstream datasets alike.

3.3 Preprocessing

Tokenizer. We trained a 32,000-token byte-level BPE tokenizer with 31,741 merges. Training used 2.0M unique words representing 3.12B word occurrences across all four Phase-1 sources. We did not lowercase or normalize Unicode: original punctuation, section symbols, party names, and citations such as "AIR 1984 SC 1728" need to remain unchanged. The tokenizer averages 5.33 characters per token and can recover the input text exactly, which is especially important when evaluation depends on character offsets.

Offsets for L-NER. We link each character to the token containing its first byte and check the offsets against the tokenizer output. Gold spans are converted to IOB2 tags and back. The first round-trip test (G1) recovered 98.13% of spans. The remaining 1.87% mainly came from 141 BPE tokens crossing an entity boundary: 130 cases ended inside a token, 16 involved whitespace, and one started inside a token. Small corrections, such as removing a final comma from a PREC span, increased the ceiling to 98.25%. A second check (G2) confirmed that window splitting and merging preserved those G1 spans across all 70 documents.

Long documents. Every downstream input begins with EOS and never

goes beyond the 1024-position pretraining limit. For NER, we use 512-token windows with a stride of 384 during training and 256 at inference, then merge overlapping predictions with centre weighting. Phases 4 and 5 use at most four 512-token chunks: two from the beginning and two from the end of a judgment. Phase 6 either combines head and tail text or produces DANCER-style summaries chunk by chunk. Phase 7 spreads up to 16 chunks of 256 tokens across each case. We tokenize only the slices needed for these layouts, using whitespace boundaries to keep token boundaries consistent and avoid slow full-document tokenization in Python. For Phase 3, the supplied sentences are used directly, with an EOS marker providing the vector for each sentence.

4 Approach and Model Evolution

4.1 Phase-1 foundation model

Bake-off. We first tested ten smaller models (29–52M parameters) on the same data before spending the full training budget (Table 2). These trial runs lasted 0.6–18.4 minutes, so they are useful for screening, not for a precise architecture ranking. Still, a clear pattern emerged. The encoder–decoder models and LSTM had much higher continuation perplexity (391–20,853) than the decoder-only transformers (205–259). That is reasonable because Phase 1 asks the model to predict text from left to right. RWKV reached the lowest perplexity, 118, but required 18.4 minutes rather than roughly 0.6 minutes for a decoder-only transformer—around 30 times longer. It was therefore not practical at our target size under the 9-hour limit.

We selected four decoder-only architectures for the full runs. They let us compare learned absolute positions, RoPE and ALiBi, along with full attention, grouped-query attention, and a sliding attention window.

Table 2: First bake-off (short runs; compare with caution). Sorted by BERTScore on the local prompts.

Architecture	Params (M)	Eval loss	PPL	BERTScore	ROUGE-L	Train min
RWKV (linear attention)	50.13	4.77	117.7	0.806	0.201	18.4
Decoder-only, GPT-2 style	30.04	5.56	258.8	0.794	0.205	0.6
Llama style	48.04	5.47	236.5	0.794	0.197	0.7
Mistral style (sliding window)	48.04	5.43	227.7	0.793	0.199	0.6
BLOOM style (ALiBi)	29.95	5.32	204.7	0.791	0.194	0.6
Encoder–decoder (FNet)	48.71	5.97	390.6	0.782	0.202	1.2
Encoder–decoder (Longformer)	51.66	6.15	468.6	0.771	0.207	0.9
Encoder–decoder (BERT2GPT2)	50.33	6.23	510.1	0.756	0.198	0.7
LSTM	32.59	6.68	797.7	0.748	0.210	0.8
Encoder–decoder (T5 style)	29.33	9.95	20,852.6	0.742	0.154	1.0

Final foundations. All four use width $d = 768$, 12 heads, a 32k vocabulary with tied input/output embeddings, pre-normalisation, and are trained on 1024-token blocks. M1 (GPT-2 style) has 14 layers, learned absolute positions (2048 slots), LayerNorm and a GELU MLP of width $4d$ (125.38M parameters). M2 (Llama style) has 12 layers, RoPE ($\theta = 10^4$), grouped-query attention with 4 key–value heads, RMSNorm and a SwiGLU MLP of width 3072 (128.40M). M3 is M2 with a 512-token sliding attention window (128.40M), so M2 versus M3 isolates the window. M4 (BLOOM style) has 14 layers, no position embedding and per-head ALiBi slopes $2^{-8h/H}$ (123.81M). Parameter counts were computed from the instantiated modules (all inside 50M–300M); each model passed causality and KV-cache tests and started within 2% of $\ln 32000$.

Objective and decoding. Training minimises next-token cross-entropy, $\mathcal{L} = -\frac{1}{T}\sum_t \log p_\theta(x_t \mid x_{<t})$, over the natural source mixture with documents separated by EOS. For submission each model

generates using the last 736 prompt tokens, except M4, which uses 1760 because ALiBi can extend beyond the training context. Each output is trimmed to exactly 128 words. On the 47 local prompt–continuation pairs, we tested sampling with $(T, p) = (0.7, 0.9)$ and $(0.4, 0.95)$, sampling with $T = 0.8$, $p = 0.95$ and a 1.1 repetition penalty, and greedy decoding with a 1.2 penalty and no repeated 4-grams. We selected the best setting for each model (Table 4).

4.2 Transfer protocol (Phases 2–7)

All downstream runs start from a Phase-1 checkpoint, loaded with exact key matching. We fine-tuned each of the four foundations for every phase so that comparisons use the same starting models. For the classification and retrieval encoders (Phases 2–5 and Phase 7), we switch from causal to bidirectional attention so the model can read context on both sides. M3 uses a symmetric 512-token window and M4 uses symmetric ALiBi distances. Phase 6 keeps causal attention because it generates summaries. Within each phase, the head, chunk layout, development split and decoding setup stay fixed; only the adaptation method changes. Our PyTorch methods include full and partial

fine-tuning (top 6 blocks, learning rate decayed by 0.85 per layer downwards), LoRA and rsLoRA, LoRMA+ (compulsory; $h = (I + sBA)W_0x + b$ with $B = 0$ at initialisation), AdaLoRA, Houlsby and Pfeiffer adapters, IA3, QLoRA-style NF4 quantisation, prefix and prompt tuning, BitFit, and knowledge distillation from a fully fine-tuned M4 teacher into students made of every second layer of each foundation (74–77M parameters), with loss $\frac{1}{2}\,\mathrm{CE} + \frac{1}{2}\,T^2\,\mathrm{KL}(p_T^{\text{teacher}} \,\|\, p_T^{\text{student}})$ at $T = 2$. Figure 2 shows how the foundation models are reused. For the downstream tasks, we only have the final Codabench scores in this report, so the discussion compares tasks rather than claiming a winner among fine-tuning methods. The two text-generation tasks scored highest: 0.829 BERTScore for continuation and 0.845 for summarization. Binary judgment prediction reached 0.735 macro-F1. Tasks requiring exact spans or less common outputs scored lower: 0.379 for statutes, 0.395 for retrieval, 0.469 for the 13 rhetorical roles, and 0.533 for strict NER. Phase 6 also had the best rank, 44, compared with ranks 60–64 for Phases 2, 4, 5 and 7; Phase 1 ranked 94 after four submissions. A possible reason summarization performs well is that it uses the same causal next-token objective as pretraining. The other tasks need new prediction heads trained from relatively small labelled datasets.

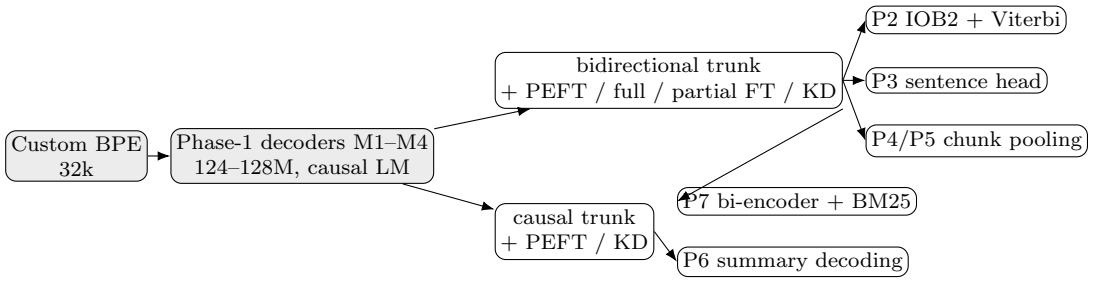


Figure 2: One tokenizer, four foundations, separately fine-tuned descendants per phase (all four foundations in every phase; the best of four is submitted).

4.3 Phase 2 - Legal NER

We use 25 IOB2 tags for token classification, then convert predicted token spans back to character offsets. This is appropriate because entities are mostly short and do not overlap; the G1 test puts the offset-related ceiling loss at 1.75%. The head consists of 0.1 dropout and a linear layer, with a CRF head also tested. At inference we use constrained IOB2 Viterbi

decoding, which does not allow an I-tag without the matching B-tag. We tune an O-shift bias on development data to balance precision and recall. To reduce false positives in repeated letterhead text, training retains 30% of windows without entities. On all four foundations, we compared full FT, LoRMA+, LoRA, adapters, partial FT and KD, using strict macro-F1 over all 12 labels to select models.

4.4 Phase 3 - Rhetorical role prediction

We process neighbouring sentences together, allowing the model to use context before and after each sentence. When a sentence appears in more than one window, we keep the prediction from the window where it has the most surrounding text. The standard head applies LayerNorm, dropout and a linear layer to the EOS-marker sentence vectors. We also tried seqctx, which adds a one-layer BiGRU to learn patterns in the sequence of roles, such as Fact, Issue, Argument, Ratio and Ruling. CL and IT were trained together, but we fitted separate class biases for each domain with two-fold cross-fitting. A bias was used only when it improved the held-out result. A one-epoch LoRA pilot on M2 compared causal and bidirectional attention; its result is stored in PILOT.json, and the default without a pilot is bidirectional attention. We tested LoRMA+, LoRA, rsLoRA ($r = 32$, $\alpha = 16$), IA3, AdaLoRA (rank 12 to 8), KD, QLoRA, prefix and prompt tuning (20 virtual tokens), and full fine-tuning.

4.5 Phase 4 - Court judgment prediction

Each judgment is represented by up to four chunks: the first two and the last two. The last hidden states within a chunk are averaged using a mask; learned attention pooling then combines the chunk vectors into one document representation for binary classification. If memory allows only one chunk, we keep the last one because judgments usually state the outcome near the end. Training uses square-root inverse-frequency class weights. We tune the threshold τ for $p(y = 1)$ between 0.05 and 0.95 on development data, while ensuring that both classes receive at least 5% of the predictions. We also remove any training or development case whose identifier or text hash matches a test case. The layout comparisons cover head–tail, first-k, last-k, and a section-based choice using ORDER/HELD headings.

4.6 Phase 5 - Legal statute identification

The Phase-4 encoder is reused with L independent sigmoid outputs, one per statute seen in training, whose biases are initialised to each statute's training log-odds. The loss is per-label BCE with $w_\ell^+ = \mathrm{clip}(\sqrt{n_\ell^- / n_\ell^+}, 1, 30)$; focal and asymmetric losses are ablations. A global threshold is tuned on development macro-F1; per-label thresholds (labels with at least 5 development positives, shrunk halfway to the global value) are used only if two-fold cross-fitting shows a gain, and every document receives between 1 and $k_{\max}$ statutes, where $k_{\max}$ is the 99th percentile of labels per training document. A "cite" layout adds the window densest in statute citations to the first and last chunks.

4.7 Phase 6 - Abstractive summarization

We reused the Phase-1 causal language models for summarization rather than adding an encoder–decoder. The input is the document followed by "SUMMARY:", with an EOS token at the start, and training loss is applied only to summary tokens. Long judgments use the beginning and end of the text, or DANCER-style chunk summaries aligned by sentence when the median summary is above 350 tokens. The output cap is selected on development data: 0.5, 0.75, 1 or 1.25 times the median training-summary length, or no cap. We compared LoRMA+, LoRA, AdaLoRA, Houlsby and Pfeiffer adapters, and KD

(online KL on summary tokens), plus QLoRA, prefix and prompt tuning and greedy versus beam decoding. No alignment or post-training method was used.

4.8 Phase 7 - Prior Case Retrieval (PCR)

For prior-case retrieval, we use the foundation as a shared-weight bi-encoder. Each chunk is mean-pooled, projected through an identity-initialized layer and L2-normalized; the average of its chunks becomes the case embedding. Training uses symmetric InfoNCE with a learned temperature, drawing negatives both from the batch and from hard BM25 matches. Other genuinely relevant cases are masked as negatives. GradCache makes larger contrastive batches possible on the T4 GPUs. At inference, we combine embedding similarity with BM25 using a weight of at least 0.2; k_1, b and bigrams are tuned on development data. We return the top seven cases and record zero-shot Phase-1 embeddings as a baseline.

5 Experiments and Implementation Details

5.1 Compute and training protocol

Training was carried out in Kaggle notebooks on two NVIDIA T4 GPUs (16 GB each). We used fp16 autocast and a gradient scaler because T4 does not support bf16. Each Phase-1 model used one GPU, with runs spread across separate accounts. The optimizer was AdamW, $\beta = (0.9, 0.95)$, weight decay 0.1, peak learning rate 3×10^{-4}, 500 warm-up steps, and cosine decay. A micro-batch of 4×1024 tokens was accumulated over eight steps, giving 32,768 tokens per update. We clipped gradients at 1.0 and used no dropout.

Checkpoints were saved every 500 steps to a private model repository, which allowed runs to resume after Kaggle stopped a session. We checked validation loss every 1,000 steps but did not stop early because it was still improving. Downstream training also used AdamW, with weight decay 0.01 (none for 1-D tensors), 6% warm-up, linear decay and clipping at 1.0. Each GPU ran one job, with atomic last.pt checkpoints every 30 minutes and early stopping after 2–3 development evaluations without improvement. M1 took 7.7 hours and M2–M4 roughly 9 hours. The downstream jobs used a time limit of a few GPU-hours (two hours by default for Phase 7), adjusting chunks and epochs to measured throughput.

Table 3: Hyperparameters of the final model in each phase. P2–P7 values are the shared defaults of the pipeline; Values are the pipeline defaults used for the runs; epochs are upper bounds with early stopping (patience 2), and Table 5 names the submitted result per phase.

Hyperparameter	P1	P2	P3	P4	P5	P6	P7
Parameters (M)	123.8–128.4	74–128	74–128	74–128	74–128	74–128	74–128
Context length	1024	512	window	4×512	4×512	≤ 1024	16×256
Peak LR	3e−4	FT 5e−5, PEFT 1e−3	PEFT 1e−3	FT 3e−5, PEFT 1e−3	as P4	PEFT 1e−3	FT 3e−5, PEFT 5e−4
Batch × accum.	4×8 blocks	32 chunks	64 sent./32 windows	16 docs	16 docs	16 docs	16 pairs
Epochs / steps	10.85k–12k steps	6 (FT), 10 (PEFT)	6 (FT), 10 (PEFT)	4 (FT), 6 (PEFT)	4 / 6	3 (FT), 4 (PEFT)	4 (FT), 5 (PEFT)
Dropout	0	0.1 (head)	head	head	head	0	0
Decoding / threshold	Table 4	Viterbi + O-shift	dev biases	τ on dev	global/per-label	word cap	top-7, fusion

5.2 Local evaluation

Phase 1 used the 47 held-out prompt–continuation pairs and our own implementations of the three metrics; Phase 2 used the 10-document development split; Phases 3–7 used the official development files, with Phase 3 bias fitting and Phase 5 per-label thresholds cross-fitted inside development so that the selection score is not inflated by the tuning. Every submission was checked by a validator before upload (only `prediction.json` at the zip root, exactly the test identifiers, correct types, integer offsets with $0 \leq s < e \leq |\text{text}|$, non-overlapping spans, unique statute identifiers, seven

retrieved cases). For Phase 1 the local estimate on 47 pairs (0.824–0.830) bracketed the Codabench score (0.8287), so the small local set was a usable proxy.

5.3 Ablations and controlled comparisons

Phase 1 provides two fairly controlled comparisons (Table 4 and Figure 3). M2 and M3 share the same architecture except that M3 has a 512-token sliding attention window. At 10,000 steps, their losses were almost identical: 1.8650 for M2 and 1.8662 for M3, a gap of about 0.001 nats. M3 also processed 2.5% fewer tokens per second. One explanation is that the removed context is not especially useful when the training sequence is only 1024 tokens long.

M1 and M4 have the same depth, width and MLP design, although their position methods and some normalization details differ. After 10,000 steps, ALiBi (M4) reached 1.8527 loss versus 1.9051 for learned absolute positions (M1), a difference of 0.052 nats. M4 had already beaten M1's final 12,000-step result by step 9,000. Speed changes the overall comparison: M1 processed 14.2k tokens/s, whereas the others managed 11.0–11.4k. It was therefore the only model to complete 12,000 steps within nine hours and finished at 1.887, close to M4's 1.853. We do not have a separate controlled downstream ablation to report. Section 4 describes those design choices, but the available results do not isolate their effects.

6 Results

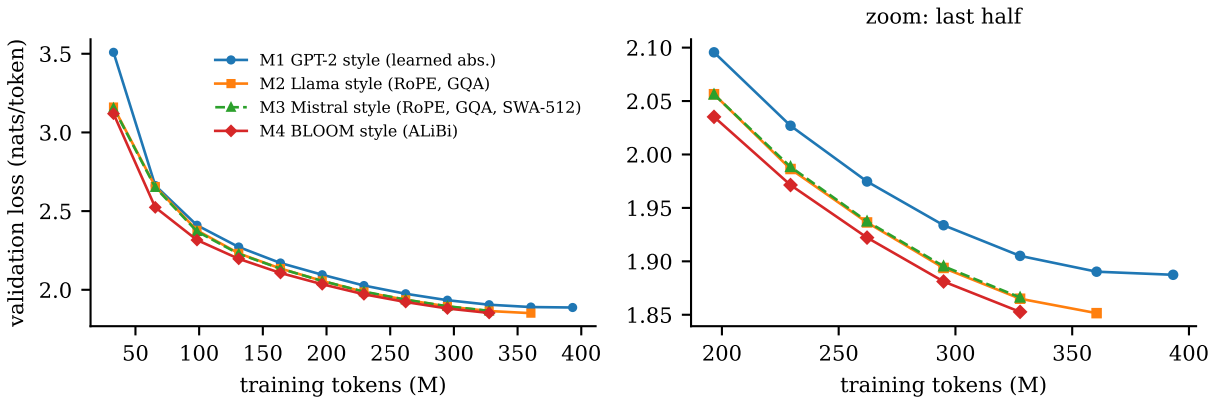


Figure 3: Phase-1 validation loss against training tokens (32,768 tokens per step). Three runs stopped at the 9-hour limit before 12,000 steps.

Table 4: Phase-1 foundations: validation loss, throughput and local scores on the 47 held-out pairs with the selected decoding.

Model	Params	Steps	Val loss @10k	Final val loss	tok/s	BERTScore	ROUGE-L	Decoding
M1 GPT-2	125.4M	12,000	1.905	1.887	14.2k	0.8239	0.1888	greedy, rep. 1.2, no-4gram
M2 Llama	128.4M	11,150	1.865	1.852 (11k)	11.4k	**0.8296**	0.1951	T 0.8, p 0.95, rep. 1.1
M3 Mistral	128.4M	10,950	1.866	1.866 (10k)	11.1k	0.8266	0.1832	greedy, rep. 1.2, no-4gram
M4 BLOOM	123.8M	10,850	**1.853**	1.853 (10k)	11.0k	0.8295	**0.1960**	T 0.8, p 0.95, rep. 1.1

Phase 1 ended in a near tie: the four foundations span 0.035 nats of validation loss and 0.006 BERTScore on the local pairs, and the 47-pair set is too small to separate M2 from M4 (difference 0.0001). The Codabench result for the submitted Phase-1 models is in Table 1 The best Codabench

score (0.8287) lies within 0.001 of the best local scores (M2 0.8296, M4 0.8295), which is consistent with the near tie between the foundations.. Decoding mattered more than architecture on the local set: for M2 the best sampling setting beat the worst by 0.026 BERTScore, four times the spread between architectures. Table 5 lists the per-phase variants.

Table 5: Per-phase variant comparison (local development metric and Codabench primary metric).

Phase	Variant (technique, foundation)	Local dev	Codabench primary
1	M1–M4 (Table 4)	0.8239–0.8296	score
2	best submission (see README)	–	0.5325
3	best submission	–	0.4692
4	best submission	–	0.7348
5	best submission	–	0.3791
6	best submission	–	0.8448
7	best submission	–	0.3952

Phase 6 gave the strongest rank in the cohort (44). The clearest controlled finding, however, comes from Phase 1, where we changed one main factor at a time under the same token budget. ALiBi reduced validation loss relative to learned absolute positions, while a 512-token sliding window made little difference. We cannot identify the best downstream fine-tuning method from the results included here because the individual development logs are not reported. The README submission table records the model and method submitted for each phase.

7 Error Analysis and Discussion

7.1 Phase-wise failure modes

Phase 1. M1 produced 20 of the 1,000 test continuations with fewer than 128 words (the shortest had 28); the other models produced 2–9 such outputs. The no-repeat-4-gram rule and 288-token generation cap sometimes ran out of space when citations used many tokens. M4 also repeated its prompt once. On the first local example, all four models reproduced the style of the Bangalore City Civil Court Act, including subsection numbering and phrases such as "shall have jurisdiction". But they also invented details: M3 mentioned "the City of Nagpur" and "Court of Small Cause at Bombay", neither present in the prompt. This suggests that a 128M model trained on 0.36B tokens can learn the form of legal writing before it learns to reproduce legal facts reliably.

Phase 2. The tokenizer's exact-span ceiling was 98.25%, much higher than the 0.533 Codabench strict macro-F1. Most errors therefore come from entity recognition, not offset reconstruction. Rare labels are a likely issue: counsel and witness labels each have fewer than 200 examples compared with 1,575 dates, and strict scoring gives no credit when a correctly identified entity has slightly wrong boundaries.

Phases 3–7. The following are likely error sources, not separately measured effects. In Phase 5, the long tail of rare statutes may explain the low 0.379 macro-F1. Phase 4 can miss a decision stated in the middle of a judgment because it reads only the first two and last two chunks. In Phase 3, a short sentence window may not distinguish a lower-court ruling from the ruling in the current judgment.

7.2 What the metrics do not measure

BERTScore and ROUGE-L reward embedding and n-gram similarity to one hidden reference, not legal correctness: the invented "Court of Small Cause at Bombay" continuation still scored well

because its vocabulary matches statutory prose. Strict span-F1 gives zero credit to a correctly typed precedent whose span loses a trailing period, while a loose metric would count it. Macro-F1 over statutes rewards guessing a rare statute once as much as finding a frequent one many times. A firm would still need citation verification and human review of generated summaries.

7.3 Limitations and what you would do next

Compute was the main limitation. Each foundation saw only about 9% of the available corpus and roughly three tokens per parameter. A 1024-token context also meant that longer judgments had to be divided into chunks. For Phases 4–7, the model then had to combine those chunks through pooling, using relatively little labelled data to learn how. We did not conduct human legal evaluation. We also trained each phase separately, which prevents errors from spreading between tasks but misses useful links, such as using Phase-2 statute spans in Phase 5. A useful next step would be to train M4 on 1B tokens and check whether ALiBi retains its advantage. We would also test 2048-token pretraining, add Phase-2 STAT spans as Phase-5 features, and use late interaction between chunk vectors in Phase 7.

7.4 Surprising findings

We expected the Llama-style model to lead because RoPE, GQA, SwiGLU and RMSNorm are common in recent architectures. Instead, the simpler ALiBi model with a GELU MLP performed just as well at comparable steps: 1.853 loss versus 1.865. Even M1, the older GPT-2-style model, came within 0.035 of the best final loss because its throughput was about 25% higher and it could train for more steps within the nine-hour limit.

8 Conclusion

The most useful Phase-1 differences were the choice of position method and the decoding setup. At equal token counts, ALiBi achieved about 0.05 nats lower validation loss than learned absolute positions. Decoding choices changed local BERTScore by as much as 0.026. By contrast, the 512-token sliding window did not produce a clear gain, and the modern foundation variants performed very similarly.

For the later tasks, the shared design decisions mattered to how we built the system: bidirectional attention for classification and retrieval encoders, chunking to stay within the 1024-token context, and development-tuned thresholds, class biases and constrained Viterbi decoding. We cannot assign an exact improvement to any one of these choices from the available leaderboard results.

Overall, training time was the bigger obstacle than a lack of text. Each of the four models used less than a tenth of the corpus yet finished within about 2% of the others in loss. For a small in-house legal language model, reliable checkpointing, faster training and careful evaluation proved at least as important as choosing a more complex architecture.

LLM Usage Declaration

Q1. Claude (Anthropic): Claude Sonnet 5.5 and Claude Opus 5.5 .

Q2.

Part	Used LLM?	Purpose of using the LLM
Understanding concepts / papers	Yes	Explaining LoRMA, AdaLoRA, ALiBi, GradCache and the task metrics
Ideation / brainstorming at a high level	Yes	Discussing candidate techniques and document layouts
Debugging small ancillary snippets	Yes	Fixing environment, padding and device errors
Plotting / visualization code	Yes	Loss-curve and label-distribution plots
Grammar / writing style of the report	Yes	Wording and grammar
Report structure / section wording	Yes	Organising the template sections and tables
Designing the model architecture	Yes	Drafting layer configurations of the four chosen families
Hyperparameter search / tuning	Yes	Proposing default learning rates, ranks and schedules
Analysis of results / conclusions	Yes	Drafting the analysis from my logged numbers
Writing the core training / inference code	Yes	Generating the PyTorch modules and notebooks, which I ran, tested and debugged
Other (specify)	No	–

Q3. Yes: the official capstone document, its evaluation scripts and the dataset card. The method descriptions were obtained through the LLM, not by reading the original papers.

Q4. I began by testing ten smaller architectures and then selected four decoder-only models that differed mainly in their positional encoding and attention patterns. That gave me a practical way to compare the designs without changing everything at once. I trained the four models from scratch on separate Kaggle accounts, working within the nine-hour limit and resuming from saved checkpoints when sessions ended. I then fine-tuned all four foundations for the later tasks and made 73 valid submissions across the seven phases.

Three of the four Phase-1 runs stopped before 12,000 steps, so I compared their results at equal token counts instead of comparing only the final checkpoints. The T5-style model did poorly in the short bake-off (perplexity 20,853), which helped justify using decoder-only models for continuation. I also found that greedy decoding left 20 out of 1,000 Phase-1 outputs shorter than 128 words, so I compared four decoding settings for each model.

One Phase-4 run could not start because the tokenizer file was missing from the Phase-1 model folder. After that, I checked model inputs more carefully before longer jobs. I also fixed environment and device issues while running the notebooks and used the Phase-1 logs to interpret the outcomes. I used Claude for generated code, report structure and draft wording, as declared above.